

IFTS N° 29

Tecnicatura Superior en Desarrollo de Software

Programación Sobre Redes

PFO 3

Rediseño como Sistema Distribuido

Arquitectura Cliente-Servidor con Sockets y Workers

Autor:

Jorge Kalas

Docente:

Germán Ríos

Primer Semestre 2026

Índice

1. Introducción	3
2. Objetivos del Proyecto	4
3. Arquitectura del Sistema	5
3.1 Clientes	5
3.2 Balanceador de Carga	5
3.3 Servidor Principal (Dispatcher)	5
3.4 Cola de Mensajes	5
3.5 Pool de Workers	5
3.6 Almacenamiento Distribuido	5
4. Tecnologías Utilizadas	6
5. Implementación del Servidor	7
5.1 Inicialización de la Base de Datos	7
5.2 Pool de Workers	7
5.3 Servidor Principal y Cola de Mensajes	7
6. Implementación del Cliente	9
6.1 Envío de Tareas	9
6.2 Comandos Disponibles	9
7. Base de Datos	10
7.1 Esquema de la Tabla tareas	10
8. Instrucciones de Ejecución	11
8.1 Requisitos	11
8.2 Clonar el Repositorio	11
8.3 Iniciar el Servidor	11
8.4 Ejecutar el Cliente	11
9. Pruebas Realizadas	12
9.1 Tarea eco — Respuesta directa del worker	12
9.2 Tarea mayusculas — Transformación de texto	12
9.3 Tarea invertir — Inversión de cadena	13
9.4 Concurrencia — Múltiples clientes simultáneos	13
9.5 Persistencia en base de datos	15
10. Respuestas Conceptuales	16
10.1 ¿Por qué usar una cola de mensajes en una arquitectura distribuida?	16
10.2 ¿Qué ventaja tiene el pool de workers respecto a crear un hilo por conexión?	16
10.3 ¿Por qué SQLite es adecuado para esta implementación académica?	16
11. Enlace al Repositorio	17
12. Conclusión	18

1. Introducción

El presente trabajo práctico tiene como objetivo transformar un sistema cliente-servidor básico en una arquitectura distribuida, haciendo uso de sockets TCP/IP, un pool de workers con hilos y una cola de mensajes interna para la distribución de tareas.

El diseño conceptual del sistema contempla los componentes típicos de una arquitectura de producción real: balanceador de carga (Nginx/HAProxy), cola de mensajes (RabbitMQ), almacenamiento distribuido (PostgreSQL, Amazon S3) y múltiples servidores worker. La implementación académica desarrollada en Python recrea esta lógica utilizando herramientas nativas del lenguaje, permitiendo ejecutar y demostrar el comportamiento del sistema sin requerir infraestructura adicional.

Este documento describe la arquitectura diseñada, las decisiones técnicas adoptadas, el código implementado y las pruebas de funcionamiento realizadas.

2. Objetivos del Proyecto

El desarrollo de este trabajo permite alcanzar los siguientes objetivos:

- ✓ Diseñar una arquitectura distribuida cliente-servidor con todos sus componentes.
- ✓ Implementar un servidor en Python que reciba tareas por socket y las distribuya a un pool de workers.
- ✓ Implementar un cliente en Python que envíe tareas y reciba resultados.
- ✓ Aplicar el patrón productor-consumidor mediante una cola de mensajes interna.
- ✓ Comprender el flujo de comunicación en sistemas distribuidos con múltiples capas.
- ✓ Documentar la diferencia entre la arquitectura objetivo de producción y la implementación académica.

3. Arquitectura del Sistema

El sistema está diseñado en capas bien definidas. El diagrama completo se encuentra disponible en el repositorio y fue creado en draw.io. A continuación se describe cada capa:

3.1 Clientes

La capa de clientes representa los distintos actores que pueden enviar tareas al sistema. En la arquitectura objetivo se contemplan clientes móviles y clientes web. En la implementación desarrollada, el rol del cliente es cumplido por `cliente.py`, un programa de consola en Python que establece conexiones TCP directas con el servidor.

3.2 Balanceador de Carga

En la arquitectura objetivo, un balanceador como Nginx o HAProxy recibe todas las solicitudes entrantes y las distribuye entre múltiples instancias del servidor para garantizar alta disponibilidad y escalabilidad horizontal. En la implementación académica, el propio servidor actúa como dispatcher al encolar las tareas recibidas.

3.3 Servidor Principal (Dispatcher)

Implementado en `servidor.py` mediante un socket TCP que escucha en `127.0.0.1:9000`. Su responsabilidad es aceptar conexiones de clientes, recibir la tarea enviada y depositarla en la cola de mensajes para que sea consumida por un worker disponible.

3.4 Cola de Mensajes

En la arquitectura objetivo se utiliza RabbitMQ como broker de mensajes, que permite desacoplar completamente el productor (servidor) de los consumidores (workers) y escalar cada componente de forma independiente. En la implementación desarrollada, esta funcionalidad es provista por `queue.Queue` de la biblioteca estándar de Python, que implementa la misma semántica FIFO de forma thread-safe.

3.5 Pool de Workers

Cuatro hilos independientes (threads) corren en paralelo, consumiendo tareas de la cola. Cada worker procesa la tarea asignada, persiste el resultado en la base de datos y devuelve la respuesta al cliente a través de la conexión socket que fue preservada. Este patrón es equivalente al de un servidor con múltiples procesos worker en producción.

3.6 Almacenamiento Distribuido

En la arquitectura objetivo se contempla PostgreSQL para datos relacionales y Amazon S3 para almacenamiento de archivos. En la implementación académica se utiliza SQLite (`tareas.db`), que provee las mismas capacidades SQL sin requerir un servidor externo. Cada tarea procesada queda registrada con su resultado, estado, IP del cliente y timestamp.

4. Tecnologías Utilizadas

Tecnología	Descripción / Rol en el sistema
Python 3	Lenguaje base del proyecto para servidor, workers y cliente
socket (stdlib)	Comunicación TCP/IP entre cliente y servidor
threading (stdlib)	Pool de workers — cada hilo procesa tareas en paralelo
queue.Queue (stdlib)	Cola de mensajes interna (equivalente académico a RabbitMQ)
SQLite / sqlite3	Persistencia de tareas y resultados (equivalente académico a PostgreSQL)
JSON (stdlib)	Serialización de mensajes entre cliente y servidor
RabbitMQ (diseño)	Cola de mensajes en arquitectura objetivo de producción
PostgreSQL (diseño)	Base de datos relacional en arquitectura objetivo de producción
Nginx / HAProxy (diseño)	Balanceador de carga en arquitectura objetivo de producción
Amazon S3 (diseño)	Almacenamiento distribuido de archivos en arquitectura objetivo

5. Implementación del Servidor

El servidor se encuentra en `servidor.py` y se compone de cuatro módulos funcionales: inicialización de la base de datos, lógica de procesamiento de tareas, workers y servidor principal.

5.1 Inicialización de la Base de Datos

Al arrancar el servidor se ejecuta `inicializar_base_de_datos()`, que crea la tabla `tareas` si no existe. Utiliza un `threading.Lock` para garantizar acceso seguro desde múltiples workers simultáneos.

```
def inicializar_base_de_datos():
    with db_lock:
        conexion = sqlite3.connect(DB_NAME)
        cursor = conexion.cursor()
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS tareas (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                cliente_ip    TEXT      NOT NULL,
                tarea         TEXT      NOT NULL,
                resultado      TEXT,
                estado        TEXT      DEFAULT 'pendiente',
                fecha         TEXT      NOT NULL
            )
        ''')
        conexion.commit()
```

5.2 Pool de Workers

Al iniciar, el servidor crea `NUM_WORKERS=4` hilos que corren la función `worker()`. Cada hilo espera bloqueado en la cola hasta que llega una tarea, la procesa, guarda el resultado en SQLite y responde al cliente.

```
def worker(worker_id):
    while True:
        item = cola_de_tareas.get(timeout=1)
        if item is None:
            break # Señal de apagado
        conn_cliente, addr, tarea = item
        resultado = procesar_tarea(tarea)
        guardar_tarea(addr[0], tarea, resultado, 'completada')
        respuesta = json.dumps({
            'worker': worker_id,
            'resultado': resultado,
            'timestamp': datetime.now().isoformat()
        })
        conn_cliente.sendall((respuesta + '\n').encode('utf-8'))
        conn_cliente.close()
```

5.3 Servidor Principal y Cola de Mensajes

La función `iniciar_servidor()` levanta el socket TCP, arranca el pool de workers y comienza a aceptar conexiones. Cada conexión entrante deposita la tarea en la cola de mensajes para ser consumida por el próximo worker libre.

```
# Configuración del socket TCP/IP
servidor_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor_socket.bind((HOST, PORT))
servidor_socket.listen(10)

# Dispatcher: encola la tarea recibida
def aceptar_conexiones(servidor_socket):
    while True:
        conn, addr = servidor_socket.accept()
        datos = conn.recv(4096).decode('utf-8').strip()
        if datos:
            cola_de_tareas.put((conn, addr, datos))
```


6. Implementación del Cliente

El cliente (cliente.py) presenta un menú interactivo en consola. Cada tarea enviada abre una nueva conexión TCP con el servidor, envía el comando y espera la respuesta JSON del worker asignado. El cliente puede enviar múltiples tareas hasta que el usuario escribe 'salir'.

6.1 Envío de Tareas

```
def enviar_tarea(tarea):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        s.sendall((tarea + '\n').encode('utf-8'))
        respuesta_raw = b''
        while True:
            fragmento = s.recv(4096)
            if not fragmento:
                break
            respuesta_raw += fragmento
            if b'\n' in respuesta_raw:
                break
        return json.loads(respuesta_raw.decode('utf-8').strip())
```

6.2 Comandos Disponibles

El cliente soporta los siguientes comandos, procesados por los workers del servidor:

Comando enviado	Resultado del worker
eco: <texto>	Devuelve el texto tal cual fue enviado
mayusculas: <texto>	Convierte el texto a MAYÚSCULAS
minusculas: <texto>	Convierte el texto a minúsculas
invertir: <texto>	Devuelve el texto invertido carácter por carácter
contar: <texto>	Devuelve cantidad de caracteres y palabras
<texto libre>	El worker genera una respuesta genérica con timestamp

7. Base de Datos

Se utilizó SQLite para la persistencia de tareas y resultados. La base de datos se almacena en tareas.db y se inicializa automáticamente al iniciar el servidor.

7.1 Esquema de la Tabla tareas

Campo	Tipo	Descripción
id	INTEGER	Clave primaria, autoincremental
cliente_ip	TEXT NOT NULL	Dirección IP del cliente que envió la tarea
tarea	TEXT NOT NULL	Texto de la tarea recibida
resultado	TEXT	Resultado devuelto por el worker
estado	TEXT	Estado: 'pendiente', 'completada' o 'error'
fecha	TEXT NOT NULL	Timestamp ISO de procesamiento

8. Instrucciones de Ejecución

8.1 Requisitos

Python 3.8 o superior. No se requieren dependencias externas: todas las bibliotecas utilizadas (socket, threading, queue, sqlite3, json) son parte de la biblioteca estándar de Python.

8.2 Clonar el Repositorio

```
git clone https://github.com/jorgekalas/RedesPFO3
cd RedesPFO3
```

8.3 Iniciar el Servidor

Ejecutar primero en una terminal:

```
python servidor.py
```

El servidor inicializará la base de datos, levantará los 4 workers y comenzará a escuchar en 127.0.0.1:9000.

8.4 Ejecutar el Cliente

En una segunda terminal, ejecutar:

```
python cliente.py
```

El cliente mostrará el menú interactivo. Se pueden enviar múltiples tareas hasta escribir 'salir'.

9. Pruebas Realizadas

Se realizaron pruebas locales ejecutando el servidor en una terminal y el cliente en otra, verificando el correcto funcionamiento de cada componente del sistema.

9.1 Tarea eco — Respuesta directa del worker

Se envió el comando 'eco:Hola Redes' desde el cliente. El worker respondió correctamente con el texto tal cual fue enviado, indicando además el ID del worker que procesó la tarea.

```
Tarea > eco:Hola Redes
[Cliente] Enviando tarea al servidor...

— Respuesta del servidor —
Worker asignado : Worker-3
Tarea enviada   : eco:Hola Redes
Resultado       : ECO >> Hola Redes
Estado          : completada
Timestamp       : 2026-05-30T17:49:29.250448
```

9.2 Tarea mayusculas — Transformación de texto

Se envió 'mayusculas:programacion sobre redes'. El worker devolvió el texto completamente en mayúsculas: 'PROGRAMACION SOBRE REDES'.

```
Tarea > mayusculas:programacion sobre redes
[Cliente] Enviando tarea al servidor...

— Respuesta del servidor —
Worker asignado : Worker-4
Tarea enviada   : mayusculas:programacion sobre redes
Resultado       : PROGRAMACION SOBRE REDES
Estado          : completada
Timestamp       : 2026-05-30T17:49:40.343481
```

9.3 Tarea invertir — Inversión de cadena

Se envió 'invertir:socket'. El worker devolvió 'tekcos', confirmando el procesamiento correcto de la cadena.

```
Tarea > invertir:socket
[Cliente] Enviando tarea al servidor...

— Respuesta del servidor —
Worker asignado : Worker-1
Tarea enviada   : invertir:socket
Resultado       : tekcos
Estado          : completada
Timestamp       : 2026-05-30T17:49:48.571617
```

9.4 Concurrencia — Múltiples clientes simultáneos

Se ejecutaron tres instancias del cliente en forma simultánea enviando tareas al mismo tiempo. El servidor distribuyó las tareas entre los workers disponibles del pool, demostrando el correcto funcionamiento de la concurrencia.

```
Tarea > eco:Hola redes en concurrencia
[Cliente] Enviando tarea al servidor...

— Respuesta del servidor —
Worker asignado : Worker-1
Tarea enviada   : eco:Hola redes en concurrencia
Resultado       : ECO >> Hola redes en concurrencia
Estado          : completada
Timestamp       : 2026-05-30T18:49:35.664280
```

```
Tarea > eco:Hola redes en concurrencia  
[Cliente] Enviando tarea al servidor...
```

```
— Respuesta del servidor —  
Worker asignado : Worker-2  
Tarea enviada   : eco:Hola redes en concurrencia  
Resultado       : ECO >> Hola redes en concurrencia  
Estado          : completada  
Timestamp       : 2026-05-30T18:49:50.259037  
_____
```

```
Tarea > eco:Hola redes en concurrencia  
[Cliente] Enviando tarea al servidor...
```

```
— Respuesta del servidor —  
Worker asignado : Worker-4  
Tarea enviada   : eco:Hola redes en concurrencia  
Resultado       : ECO >> Hola redes en concurrencia  
Estado          : completada  
Timestamp       : 2026-05-30T18:49:38.183545  
_____
```

9.5 Persistencia en base de datos

Se verificó el contenido de tareas.db consultando la tabla tareas. Todas las tareas procesadas quedaron registradas con su resultado, estado 'completada' y timestamp correspondiente.

```
Pc@PC MINGW64 /d/Tecnicatura en Desarrollo de Software/05 - Quinto Semestre/01
- Programacion sobre Redes/PFO3/Codigo (main)
$ python -c "import sqlite3; conn=sqlite3.connect('tareas.db'); [print(r) for r
● in conn.execute('SELECT * FROM tareas')]; conn.close()"
(1, '127.0.0.1', 'eco:Hola Redes', 'ECO >> Hola Redes', 'completada', '2026-05-
30T17:49:29.167133')
(2, '127.0.0.1', 'mayusculas:programacion sobre redes', 'PROGRAMACION SOBRE RED
ES', 'completada', '2026-05-30T17:49:40.267604')
(3, '127.0.0.1', 'invertir:socket', 'tekcos', 'completada', '2026-05-30T17:49:4
8.153931')
(4, '127.0.0.1', 'eco:Hola Redes', 'ECO >> Hola Redes', 'completada', '2026-05-
30T17:51:22.386359')
(5, '127.0.0.1', 'eco:Hola Redes', 'ECO >> Hola Redes', 'completada', '2026-05-
30T17:51:23.574362')
(6, '127.0.0.1', 'mayusculas:hola mundo', 'HOLA MUNDO', 'completada', '2026-05-
30T17:51:56.592872')
(7, '127.0.0.1', 'invertir: Hola Mundo', 'odnuM aloH', 'completada', '2026-05-3
0T17:52:09.844234')
```

10. Respuestas Conceptuales

10.1 ¿Por qué usar una cola de mensajes en una arquitectura distribuida?

Una cola de mensajes cumple el rol de desacoplador entre productores y consumidores. Sin ella, el servidor tendría que esperar a que un worker esté libre antes de aceptar la próxima conexión, creando un cuello de botella. Al depositar cada tarea en la cola, el servidor puede seguir aceptando nuevas solicitudes de inmediato, mientras los workers consumen tareas a su propio ritmo.

Esto aporta tres ventajas fundamentales en sistemas distribuidos: (1) tolerancia a picos de demanda — la cola absorbe ráfagas de solicitudes sin perder tareas; (2) escalabilidad independiente — se pueden agregar workers sin modificar el servidor; (3) resiliencia — si un worker falla, la tarea permanece en la cola y puede ser tomada por otro worker.

RabbitMQ es el estándar de industria para este rol porque además ofrece persistencia de mensajes, garantías de entrega y enrutamiento avanzado. En esta implementación, `queue.Queue` provee la misma semántica FIFO y thread-safety de forma nativa.

10.2 ¿Qué ventaja tiene el pool de workers respecto a crear un hilo por conexión?

El patrón "un hilo por conexión" tiene un problema de escala: crear y destruir hilos tiene un costo de recursos significativo, y con alta concurrencia el sistema puede quedarse sin memoria o degradarse por el overhead de context switching. Un pool de workers fija la cantidad de hilos al arranque y los reutiliza indefinidamente, eliminando ese overhead.

Adicionalmente, el pool actúa como mecanismo de backpressure natural: si los 4 workers están ocupados, las nuevas tareas esperan en la cola sin bloquear el servidor principal. Esto protege al sistema de ser saturado por un pico de solicitudes.

10.3 ¿Por qué SQLite es adecuado para esta implementación académica?

SQLite es un motor relacional embebido que no requiere servidor externo, lo que simplifica notablemente la configuración del entorno de desarrollo y la reproducibilidad del proyecto. Toda la base de datos es un único archivo (`tareas.db`) portable junto al código fuente.

Para un sistema de producción con múltiples workers en distintas máquinas, sería necesario migrar a PostgreSQL u otro motor que soporte acceso concurrente desde múltiples procesos/nodos. En este contexto académico, el uso de `db_lock` (`threading.Lock`) garantiza acceso serializado desde los 4 workers.

11. Enlace al Repositorio

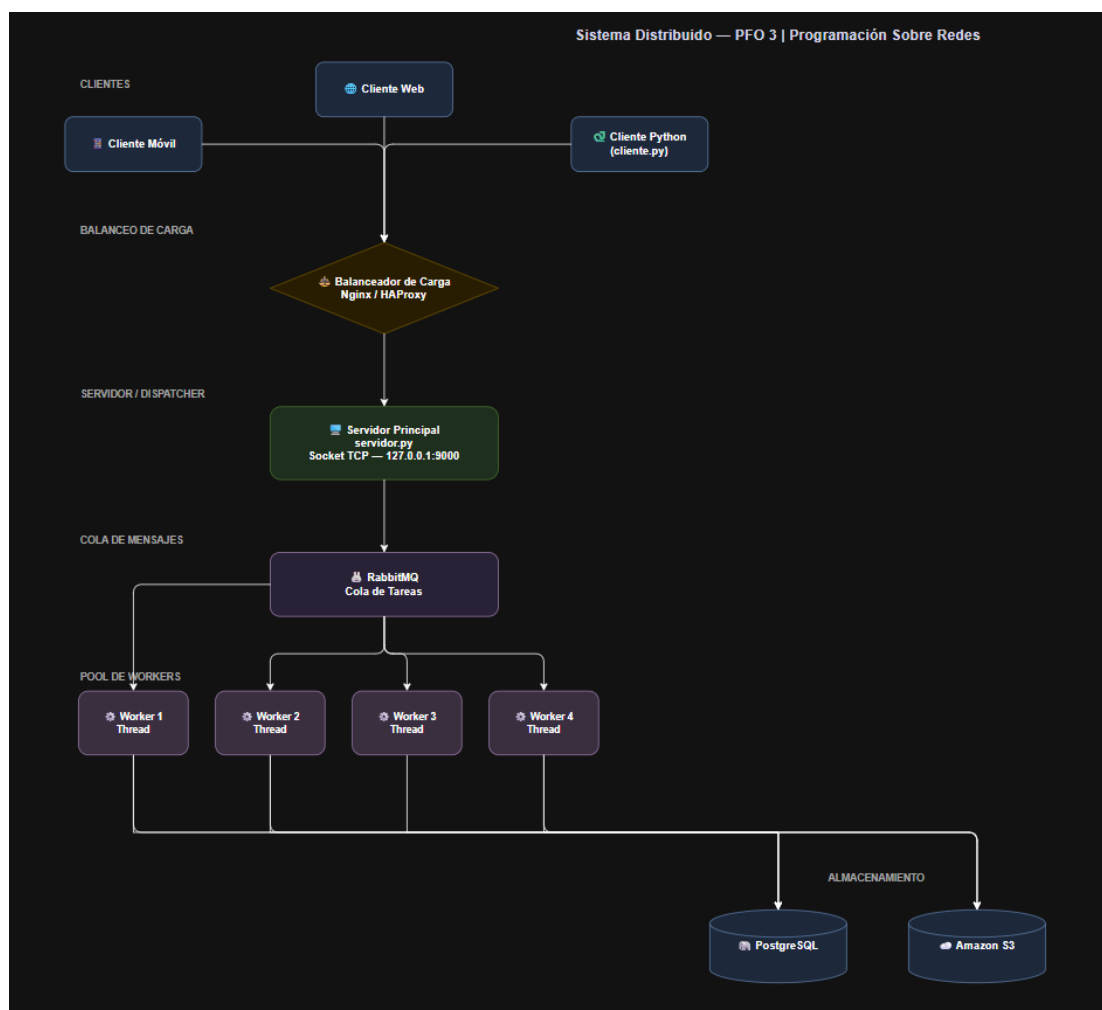
El código fuente completo, el diagrama del sistema y el presente informe se encuentran disponibles en el repositorio de GitHub:

Repositorio: <https://github.com/jorgekalas/RedesPFO3>

El repositorio incluye:

- servidor.py — Servidor con pool de workers y cola de mensajes
- cliente.py — Cliente interactivo con soporte de múltiples comandos
- tareas.db — Base de datos SQLite generada automáticamente al ejecutar
- diagrama_pfo3.xml — Diagrama de arquitectura importable en draw.io
- README.md — Instrucciones de instalación y ejecución

Asimismo, el diagrama del sistema distribuido puede consultarse en este [enlace](#) a Draw.io y a continuación se adjunta, a modo de referencia, una captura del mismo:



12. Conclusión

El desarrollo de esta PFO permitió comprender en profundidad los principios de los sistemas distribuidos, aplicando conceptos como la arquitectura por capas, el patrón productor-consumidor, la concurrencia con pools de hilos y la persistencia de datos en un contexto de múltiples workers.

La decisión de implementar la arquitectura con herramientas nativas de Python (socket, threading, queue, sqlite3) mientras se documenta la equivalencia con componentes de producción (RabbitMQ, PostgreSQL, Nginx) permite demostrar comprensión conceptual real sin sacrificar la ejecutabilidad del sistema en un entorno local.

El sistema resultante es funcional, concurrente y extensible: agregar nuevos tipos de tareas al servidor o aumentar el pool de workers requiere cambios mínimos. Esto valida el diseño como una base sólida sobre la cual escalar hacia una arquitectura de producción completa.

Jorge Kalas - IFTS N° 29 - Primer Semestre 2026